ENCM 511: Embedded System Interfacing
Assignment 4

Group 6
Chloe Fulbrook - 30210975
Ethan Lee - 30213225
Devon Calvin - 30204532

Submitted on November 7th, 2025

General Code Explanation:

Similar to previous labs, the Lab 4 code is divided into a number of files to help with the organization and readability of the project.

init.c
This file is identical to the init.c file submitted in Lab 3. It holds the initializer functions for the IOs (buttons and LEDs of the board), and configures the IOC interrupts. It also configures the timers, and creates a delay function (not used in this lab).

init.h
This is the header file that accompanies init.c. It is again unchanged from Lab 3. It defines some macros, and holds the function prototypes for the initializer functions.

uart.c
This file is almost identical to how it was in Lab 3. The differences are that we removed some logic related to a Lab 3-specific flag, and added additional logic into the U2RX ISR. This logic is used to detect the keyboard input from the user, allowing us to accept a change at any time. Essentially, it checks if the received character is one of the ones that forces a change in the display, checking ‘d’ for decimal, ‘x’ for hex, and ‘v’ to print a voltage in millivolts. These characters update the mode variable connected to an enum type, and this can be read by the function controlling the display. If another character is entered, the ISR simply completes and does not update mode, preserving the current state of mode until a valid character is entered. Otherwise, the file just includes the general UART functions.

uart.h
This is also essentially unchanged from its version in Lab 3. It holds the function prototypes for the lab 4 UART functions.

adc.c
This file holds the functions related to the ADC. It begins with the ADC1_init() function that initializes the ADC used in the lab. We initialize the ADC to 10-bit mode, set the positive and negative references to AVDD and VSS respectively, and set up the rest of the config registers. The exact specifications can be seen in the file.

The do_ADC() function obtains a single result from the ADC and stores it in a result variable. The while loop polls the AD1CON1bits.DONE to wait for the conversion to complete, and then the result gets the conversion result and returns it.

The do_ADC_avg() function takes an average of the last n samples from the ADC. This is done to help with the stability of the reading. We declare a uint32 variable sum that gets the sum of the

n ADC readings. A uint32 is used in order to protect against overflow of the sums. After this, the ADC is turned off, and the sum is divided by the number of samples taken. It is then casted back to a uint16.

adc.h
This file is the header file associated with adc.c. It simply holds the function prototypes for the functions described above.

lab4_functions.c
This file contains all of the additional functions created for Lab 4. It starts with bar_graph(), which is a function used to control the printing of the voltage bar. It takes in the result of the ADC and a char c as arguments. The number of bars to be printed is the ADC result divided by 32 to give a maximum of 32 bars. It then calls XmitUART2 to print the given char num_bars times, and prints a space afterwards. The char c is used so that we can change the look of the bar display between different symbols.

The function int_to_str_dec() is used to convert a decimal integer to a string. It takes in a decimal number and a pointer to string as arguments. It begins by checking if the number is 0, creates that string, and should end. Otherwise, we modulo off the digits, convert them to their ascii representation and append to the string, and then divide the number to move off that digit. The string then enters a while loop to reverse it, as the modulo creates the string in reverse order. We then null-terminate the string, and return.

int_to_str_hex() is similar to the above, except it converts a decimal integer into a hex string. It starts by placing the 0x signifier into the string, then pulls nibbles out of the number and indexes the string "0123456789ABCDEF" based on the nibble to place the hex digit into the string. It then null terminates the string, and returns.

adc_to_mvolt() is used to convert the raw ADC reading to a value in millivolts. We use a uint32 variable to protect against overflow, and evaluate $V = \frac{raw\ reading}{resolution} \times AV_{DD}$, with AVDD being 3000mV for this configuration. We then cast to a uint16 to return.

The last function in this file is the display_func(), used to actually display over UART. It takes in the ADC result and the mode. Depending on the mode, it sets up the write string with the mode display, and creates a string of either the raw decimal result, the hex, or the millivolt value. It then clears the terminal, prints the bar graph, the write string, and the adc result in the appropriate form.

lab4_functions.h
This is the header file accompanying the lab4_functions.c file. It has some macro definitions for the debounce time, timer period, and the voltage value. It creates the enum types for the state

machine, and declares the extern volatile mode and bar_char variables that are modified in multiple files. Then the function prototypes for all of the above functions are included as well.

lab4_main.c
This is the main file for lab 4. It begins with the #pragmas for setup and configuration, and includes the necessary headers. A number of variables are created to help with the button logic in the timer 2 ISR, same as they were in Lab3. We also set up some global variables to track a begin_flag, along with defaulting the state machine state, the mode, and the bar character. Upon entering main, we configure RB3 to analog, and call the initializer functions. We define variables to store the current and previous results from the ADC, and track the previous bar character, and we set the start and stop counts for the TMR3.

In the while(1) superloop, we enter the switch statement, and enter the begin state. To slow down the rate of sampling, the begin state only transitions to sampling when the TIMER3 goes off. If the begin flag is low, we are not currently counting, and so we start counting by setting the flag high and turning timer 3 on. Upon the 0.25s elapsing, we enter the T3 ISR, where we clear the count and interrupt flag, turn on the ADC, turn off the timer, and clear the begin flag. It then transitions the state to the sample state.

In the sample state, we call the do_ADC_avg() function to get an average result of the last 8 samples. By averaging, we are able to get a more stable result and see less fluctuation on the screen. After the ADC result has been stored, we transition to the display state.

In the display state, we check to see if a component of the display (either the adc result, the mode, or the bar character), and if it has, we update the display using the previously described function. After this, we update the previous states of the ADC result, mode, and bar character. Finally, we transition back to the begin state and cycle.

The other components of this file are the IOC ISR, which is the same as in the last lab. The T2 ISR is also the same in terms of the logic, the only difference is that instead of setting button flags, we instead update the char used to display as the bar.

Special Function(s):
Clear_terminal

```
void clear_terminal(void) {
    // using escape code to clear the terminal screen
    Disp2String("\033[2J\033[H");
}
```

This contains the ANSI escape codes to clear the terminal screen and reset the cursor to the top left. The purpose of this function is to clear the terminal when we want to display different text. Putting this before any new Disp2String call makes the terminal only display one line

bar_graph

```
void bar_graph(uint16_t adc_res, char c){
    // a function to print the bar graph based on the result of the adc
and the declared character
    uint16_t num_bars = adc_res / 32;   // divide into 32 increments,
giving a max of 32 bars of display
    XmitUART2(c, num_bars);     // char c can be used to change the bar
character
    XmitUART2(' ', 1);          // a space for display help
}
```

Can display a bar of up to 31 characters in length. It takes in the ADC reading and displays 0-31 characters for the bar depending on the magnitude. Calls XmitUART2 and prints whatever bar character a num_bars amount of times.

display_func

```
void display_func(uint16_t adc_res, mode_t mode){
    // a function that handles the actual displaying of the screen on each
update
    char *write;
    char adc_str[10];
    switch(mode){
        // switches based on the mode set by the keyboard key presses
        case(MODE_DEC):
            // setup the strings to print for the decimal mode
            write = "Decimal value: ";
            int_to_str_dec(adc_res, adc_str);
            break;
        case(MODE_HEX):
            // setup strings to print for the hex mode
            write = "Hex value: ";
            int_to_str_hex(adc_res, adc_str);
            break;
        case(MODE_VOLTAGE_DEC):
```

```
            // setup strings to print for the voltage mode
            write = "Voltage value (mV): ";
            uint16_t voltage = adc_to_mvolt(adc_res);
            int_to_str_dec(voltage, adc_str);
            break;
    } // end switch(mode)

    clear_terminal();               // clear the screen
    bar_graph(adc_res, bar_char);   // print the bar graph
    Disp2String(write);             // print the starter string
    Disp2String(adc_str);           // print the value, in whichever form
it needs
}
```

Takes in ADC reading and the mode for what type of output to produce in the terminal. Within the if statements, it sets write as the type of output it is displaying and converts the ADC reading into a string in the proper format. The terminal is then cleared to keep it on one line. The code will then write the bars, a description of how the data is interpreted, and lastly the calculated value from the ADC.

int_to_str_dec

```
void int_to_str_dec(uint16_t dec, char *str){
    int i = 0;
    if(dec == 0){
        // if the number is just zero, put that in the string and move on
        str[i] = '0';
        i++;    // dont forget to increment the index variable!
    }
    else{
        while(dec > 0){
            // while the number is more than 0, we modulo off the digits
and put them in the string
            str[i++] = (dec % 10) + '0';
            dec /= 10;
        }
    }

    int start = 0;
    int end = i - 1;
```

```
    while(start < end){
        // this loop just reverses the string so its the right order
        char temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end --;
    }
    str[i] = '\0';
    return;
}
```

Takes in the ADC reading and the address of the character array that is used for write in display_func. The purpose is to convert the integer into decimal and format it into a string. By using the %, you get the remainder and that is the last digit of the number. By adding that value to '0' you get the ASCII of that number. This is then stored at the beginning of a character array. The dec number is then divided by 10 so that this conversion can happen to the next digit. This process loops until dec is less than zero. Due to how the code was written, the number is in reverse and so to fix this the string is reversed through the following process. Firstly, the index of the end and start of the string are recorded, then the character at the start of the string is temporarily saved, then the character at the end of the string is assigned to the beginning. After that, the end is assigned the old start value and the index of start and end are incremented and decremented to swap the next characters. This process stops when start<end which means that the middle character was reached or all pairs were swapped.

int_to_str_hex

```
void int_to_str_hex(uint16_t dec, char *str){
    // this function converts an int (dec) to a hex string
    str[0] = '0';
    str[1] = 'x';

    int i = 0;
    char hex[] = "0123456789ABCDEF";    // hex string to index from
    for(i; i < 4; i++){
        uint8_t nibble = (dec >> (12 - 4 * i)) & 0x0F;  // pulls
individual nibbles from the decimal int
        str[2 + i] = hex[nibble];   // add it to the string
    }
```

```
    str[2 + i] = '\0';
    return;
}
```

Takes in the ADC reading and the address of the character array that is used for write in display_func. The purpose is to convert the integer into hex and format it into a string. It first begins the string with 0x to format it in hex. The way it works is by right shifting the integer to until the desired nibble is the 4 LSB. To obtain these bits, it is then ANDED with 00001111. This nibble is then used as the index for our hex character array and it indexes the appropriate hex value. These hex values are converted from the MSB to LSB of the integer and each result is stored in a string in order after the 0X.

adc_to_mvolt

```
uint16_t adc_to_mvolt(uint16_t adc){
    // converts the raw adc value to a voltage in mV
    uint32_t v;             // the multiplication below may overflow a
uint16, so use a uint32 and cast later
    v = adc * AVDD / 1024;  // this just converts the raw adc to a voltage
(in mV)

    return (uint16_t) v;    // casts the voltage to a uint16 (max is 3V
(3000mV) so it fits) and returns
}
```

Takes in the adc result and scales it into mV. Using the formula $\frac{ADC\ reading}{ADC\ resolution} * reference\ voltage\ in\ mV$, the ADC reading is converted to mV. This value will eventually be used int_to_str_dec.

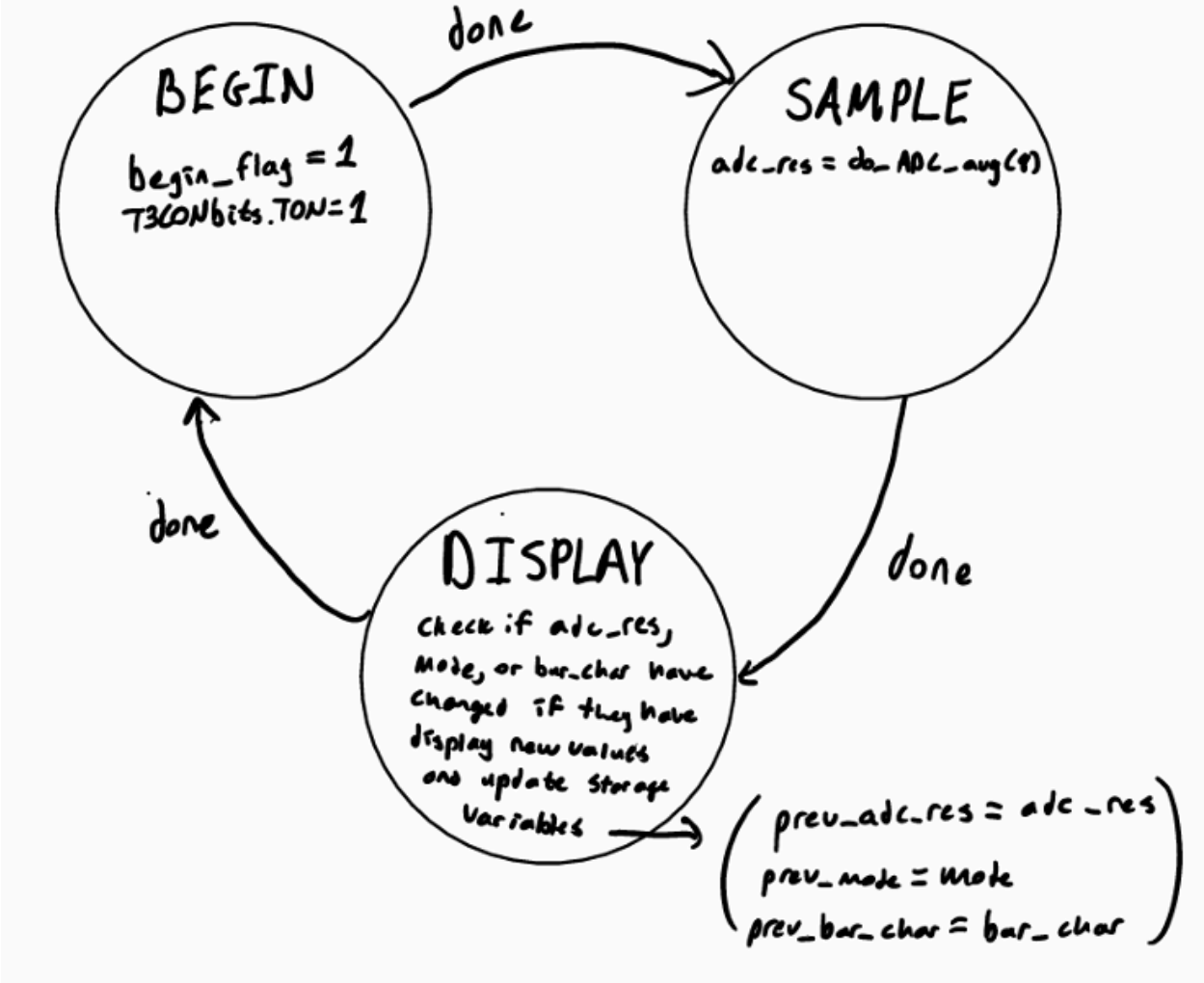
do_ADC_avg

```
uint16_t do_ADC_avg(uint8_t samples){
    uint32_t sum = 0;   // track the sum. this needs to be uint32 cause
the sum may overflow
    for(uint8_t i = 0; i < samples; i++){
        // loop through the number of samples to track
        sum += do_ADC();    // add to the sum
    }
```

```
    AD1CON1bits.ADON = 0;    // turn off the ADC when done sampling
    return (uint16_t)(sum / samples);    // divide to get the average
value, and cast back to a uint16
}
```

Reads the ADC n times and outputs the average of the samples. This produces a more stable reading as the ADC reading will fluctuate without changing anything.

Our Moore-type FSM:



Questions:

In your assignment part, how do you accept the keyboard press at any time?

To accept the keyboard press at any time, we utilize the U2RX ISR in the uart.c file. Any time something is received by the UART via the keyboard, this ISR is entered. Upon entering this ISR, we check if the received character is one of the valid characters, being ‘d’ for decimal, ‘x’ for hex, and ‘v’ for a millivolt voltage. If it is valid, we set the appropriate mode. If the received char is not one of these, nothing occurs, and the previous state is preserved.

How did you set up the ADC?

The ADC was configured the following way:

1. ANSELB |= 0b1000 which sets PIN7, AN3, as analog input
2. The positive and negative voltage reference were set to VDD and VSS respectively
3. Sampling is done on the A channel and the positive input is AN5 (PIN 7) and the negative input is VSS
4. The Sample Clock Source is set to auto-convert mode which means the conversion is done when after sampling instead of doing it based on a CPU clock
5. The ADC’s clock period is 256 * the CPUS clock period (Tad)
6. The ADC is configured to automatically sample when the last conversion is finished
7. The ADC automatically samples for 31 * Tad which is fine as this application does not need fast readings
8. The ADC outputs the reading as an absolute decimal result, unsigned, right justified in the ADC buffer
9. It’s set in 10 bit mode